

**CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CAMPUS TIMÓTEO**

Marcos Vinícius de Oliveira Silva

**COMPARAÇÃO ENTRE MODELOS DE TRADUÇÃO NEURAI
BASEADOS NA ARQUITETURA TRANSFORMER DE CÓDIGO
ABERTO**

Timóteo

2026

Marcos Vinícius de Oliveira Silva

**COMPARAÇÃO ENTRE MODELOS DE TRADUÇÃO NEURAI
BASEADOS NA ARQUITETURA TRANSFORMER DE CÓDIGO
ABERTO**

Monografia apresentada à Coordenação de Engenharia de Computação do Campus Timóteo do Centro Federal de Educação Tecnológica de Minas Gerais para obtenção do grau de Bacharel em Engenharia de Computação.

Orientador: Douglas Nunes de Oliveira

Timóteo

2026

Lista de ilustrações

Figura 1 – Tradução Automática Baseada em Regras	10
Figura 2 – Tradução Automática Baseada em estatística	12
Figura 3 – Tradução automática neural	13
Figura 4 – Tokenização de uma frase em português	15
Figura 5 – Conversão de tokens em embeddings	16
Figura 6 – Conversão de pontuações em probabilidades	17
Figura 7 – Propriedade de normalização da função softmax	17
Figura 8 – Tradução sem a utilização de um mecanismo de atenção	19
Figura 9 – Tradução com a utilização de um mecanismo de atenção	19
Figura 10 – Arquitetura transformer	20
Figura 11 – Exemplo de codificação posicional aplicada a uma sequência	21
Figura 12 – Rede neural feed-forward	22
Figura 13 – Procedimento	36

Lista de tabelas

Tabela 1 – Especificações da máquina virtual utilizada	33
Tabela 2 – Exemplo de pares de tradução da base OpenSubtitles v2024	35
Tabela 3 – Exemplo de pares de tradução da base MDN Web Docs	35
Tabela 4 – Exemplo de pares de tradução da base PHP	36
Tabela 5 – Cronograma das atividades propostas	38

Lista de abreviaturas e siglas

RNN	Redes neurais recorrentes
CPU	Unidade central de processamento
RAM	Memória de acesso aleatório
BLEU	Substituto para avaliação bilíngue
SMT	Tradução Automática Estatística
RBMT	Tradução Automática Baseada em Regras
NMT	Tradução automática neural
chrF	Pontuação F de n-gramas de caracteres
NLTK	Conjunto de ferramentas para linguagem natural

Sumário

1	INTRODUÇÃO	7
1.1	Justificativa	7
1.2	Problema	8
1.3	Objetivos	8
2	EVOLUÇÃO DA TRADUÇÃO AUTOMÁTICA	10
2.1	Tradução Automática Baseada em Regras	10
2.2	Tradução Automática Estatística	11
2.3	Tradução Automática Neural	12
3	REFERENCIAL TEÓRICO	15
3.1	Tradução Automática Neural - NMT	15
3.2	Tokenização	15
3.3	Embedding	16
3.4	Função Softmax	16
3.5	Mecanismo De Atenção	18
3.6	Arquitetura Transformer	19
3.6.1	Visão Geral	19
3.6.2	Positional Encoding	20
3.6.3	Feed-Forward Neural Network	21
3.6.4	Scaled Dot-Product Attention	22
3.6.5	Multi-Head Attention	23
3.6.6	Encoder	23
3.6.7	Decoder	24
3.7	Modelos de Tradução Open Source	25
3.7.1	MarianMT	25
3.7.2	M2M-100	25
3.7.3	NLLB	26
3.8	Métricas de Avaliação	26
3.8.1	BLEU	26
3.8.2	BERTScore	27
3.8.3	chrF	28
3.9	Eficiência Computacional	28
4	PROCEDIMENTOS METODOLÓGICOS	30
4.1	Modelos	30
4.2	Tipos de texto	31
4.3	Métricas de Avaliação	31
4.3.1	BLEU	31

4.3.2	BERTScore	32
4.3.3	chrF	32
4.4	Ambiente Computacional	33
4.5	Desempenho Computacional	33
4.5.1	Uso de CPU e Tempo de Execução	33
4.5.2	Consumo de Memória RAM	34
4.6	Base de Dados	34
4.6.1	Textos Conversacionais e Informais	35
4.6.2	Textos Específicos	35
4.7	Procedimento de Execução	36
4.8	Análise dos dados	36
4.8.1	Comparação entre modelos	37
4.8.2	Comparação entre tipos de texto	37
4.8.3	Correlação entre métricas	37
5	CRONOGRAMA	38
	REFERÊNCIAS	39

1 Introdução

O mundo passou e continua passando por grandes mudanças tecnológicas nos últimos anos, o que possibilitou a presença da tecnologia em diversos setores econômicos (CLARO, 2009). Juntamente com essa presença tecnológica, as pessoas passaram a se tornar mais ativas no ambiente virtual, uma vez que a tecnologia faz parte do dia a dia. Em conjunto com essa presença digital, tornou-se notável o aumento do consumo de conteúdos em outros idiomas, bem como a realização de compras de produtos, sejam eles físicos ou digitais, também em línguas estrangeiras (BULLA, 2024).

Ferramentas de tradução são muito úteis para pessoas que não são capazes de compreender a língua que está sendo apresentada. Uma alternativa para contornar o problema da barreira linguística é a utilização de tradutores automáticos, capazes de identificar o conteúdo transmitido por meio do contexto e da forma de escrita, gerando uma tradução que se assemelhe ao máximo à língua de destino. Atualmente, existem diferentes ferramentas que realizam a tradução de uma língua de origem para uma língua de destino. Como exemplo, podem ser citadas as ferramentas Google Tradutor, DeepL, Smartling e Lokalise, todas com o mesmo objetivo: traduzir textos mantendo o contexto e o sentido da sentença original.

Até o ano de 2017, diferentes técnicas e algoritmos foram utilizados para a tradução entre línguas. Dentre as abordagens existentes, destacam-se inicialmente as técnicas baseadas em regras gramaticais (Rule-Based Machine Translation - RBMT), que representaram uma das primeiras fases das ferramentas de tradução (TORREGROSA et al., 2019). Posteriormente, surgiram os modelos baseados em métodos estatísticos (Statistical Machine Translation – SMT), que representaram um avanço significativo na tarefa de tradução, alcançando níveis mais satisfatórios e adaptativos em comparação às abordagens baseadas em regras gramaticais (LOPEZ, 2008).

Ainda antes de 2017, modelos baseados em redes neurais recorrentes (RNN) passaram a ser amplamente utilizados na tarefa de tradução, sendo bem aceitos por apresentarem resultados superiores em relação às abordagens anteriores (DANG, 2023). No entanto, em 2017, foi proposta a arquitetura Transformer, que trouxe modificações relevantes em relação aos modelos anteriores, como o uso do mecanismo de autoatenção. Esse mecanismo possibilitou avanços significativos na identificação de contexto e na análise de sequências mais longas, contribuindo para traduções mais precisas, tanto em termos de contexto quanto na captura de dependências em textos mais extensos (ANAND et al., 2025).

1.1 Justificativa

Atualmente, existem diversos modelos de tradução baseados na arquitetura Transformer e disponibilizados como código aberto. Empresas como Meta e Google, além de iniciativas como o MarianMT, que não possui fins lucrativos, estão em constante desenvolvimento e pes-

quisa na área de tradução entre línguas (FEINER; LESWING, 2025). Esse cenário evidencia que a área de tradução automática encontra-se em constante evolução, podendo trazer benefícios em escala global, especialmente para a população brasileira, na qual apenas uma parcela possui fluência na língua inglesa (OLIVEIRA, 2026). Dessa forma, avanços em modelos de tradução podem ser amplamente aproveitados por pessoas que desejam ou necessitam consumir, comprar ou até mesmo comercializar produtos e conteúdos em outros idiomas, especialmente em inglês.

Apesar da ampla disponibilidade de modelos de tradução baseados na arquitetura Transformer, sejam eles desenvolvidos por organizações com ou sem fins lucrativos, existem diferenças relevantes entre esses modelos. Tais diferenças podem estar relacionadas à forma como a arquitetura é implementada, bem como às bases de dados utilizadas durante o treinamento. Como consequência, essas variações podem impactar diretamente o comportamento dos modelos em diferentes cenários, influenciando aspectos como consumo de memória, uso de CPU e tempo de resposta, mesmo quando submetidos à mesma entrada.

Diante desse contexto, este trabalho propõe a comparação entre diferentes modelos de tradução automática, com o objetivo de analisar seus desempenhos e apresentar resultados que auxiliem na escolha do modelo mais adequado para diferentes cenários de uso, considerando a tradução entre uma língua de origem e uma língua de destino.

1.2 Problema

A seguinte questão constitui o problema desta pesquisa: Qual é o desempenho de diferentes modelos de tradução entre línguas e automática, baseados na arquitetura Transformer para a tarefa de tradução do inglês para o português, considerando métricas como consumo de memória RAM, uso de CPU, pontuação BLEU, BERTScore e chrF, em diferentes cenários de teste?

1.3 Objetivos

Para responder ao problema proposto, o objetivo geral deste trabalho é investigar e comparar o desempenho de modelos de tradução entre línguas e automática baseados na arquitetura Transformer na tarefa de tradução do inglês para o português, considerando métricas de qualidade (BLEU, BERTScore e chrF) e eficiência computacional (uso de memória RAM e CPU) em diferentes cenários experimentais.

Também, objetivam-se mais especificamente:

1. Revisão bibliográfica sobre o funcionamento de modelos de tradução automática neural baseados na arquitetura Transformer;
2. Selecionar e configurar os modelos de tradução automática neural de código aberto;
3. Selecionar base de dados pública para comparação entre os modelos de tradução automática neural;

4. Avaliar qualidade de tradução utilizando mettricas como BLEU, BERTScore e chrF;
5. Medir eficiência de recursos computacionais quando se utiliza modelos de tradução;
6. Avaliar os modelos de tradução diferentes cenários de teste;
7. Analisar correlação entre métricas de qualidade de tradução.

2 Evolução da Tradução Automática

A área de tradução automática tem passado por transformações significativas ao longo dos anos, seja pela evolução de modelos específicos por meio de técnicas de otimização, seja pela proposição de novas arquiteturas que estabeleceram novos estados da arte. Este capítulo tem como objetivo apresentar um panorama da evolução das técnicas e modelos de tradução automática, desde as abordagens iniciais até os métodos mais recentes, discutindo suas principais características, vantagens e limitações.

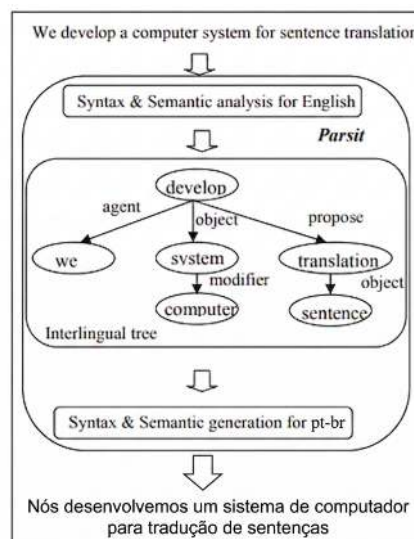
2.1 Tradução Automática Baseada em Regras

A tradução automática baseada em regras, do inglês Rule-Based Machine Translation (RBMT), consiste em sistemas que operam com base em um conjunto de regras linguísticas previamente definidas para a língua de destino. Esses sistemas utilizam regras gramaticais para realizar a tradução entre línguas, exigindo um esforço humano significativo na elaboração de todos os componentes necessários para a tradução de uma língua de origem para uma língua de destino.

Entre esses componentes, destacam-se analisadores morfológicos, etiquetadores de classes gramaticais, analisadores sintáticos, dicionários bilíngues, regras de transferência, geradores morfológicos e regras de reordenação, entre outros (S, 2017).

A Figura 1 ilustra o processo de tradução baseado em regras. Nela, uma frase em inglês é fornecida para o sistema, e baseado nas regras pre-definidas da língua portuguesa como língua de destino a tradução é gerada.

Figura 1 – Tradução Automática Baseada em Regras



Fonte: (CHAROENPORNSAWAT; SORNLETLAMVANICH; CHAROENPORN, 2002) adaptada para português

A tradução automática baseada em regras teve início na década de 1970, sendo considerada um marco na área de tradução automática e uma abordagem inicialmente promissora (WIKIPEDIA, 2026). No entanto, algumas características limitaram sua adoção em larga escala (OKPOR, 2014):

- Escassez de dicionários bilíngues de alta qualidade, sendo sua construção uma tarefa custosa;
- Necessidade de configuração manual de diversas informações linguísticas;
- Dificuldade em lidar com ambiguidade linguística e expressões idiomáticas, especialmente em sistemas de grande escala.

Devido a essas limitações, tornou-se necessária a criação de novas abordagens capazes de lidar melhor com ambiguidade, dependências contextuais entre palavras e reduzir a necessidade de intervenção humana na construção das traduções. Nesse contexto, surgiu a tradução automática baseada em Estatística (Statistical Machine Translation – SMT), que será abordada na próxima seção.

2.2 Tradução Automática Estatística

A abordagem estatística de tradução baseia-se em modelos probabilísticos extraídos de bases textuais bilíngues alinhadas, nos quais cada palavra ou frase da língua de destino é associada a elementos da língua de origem segundo uma determinada probabilidade. Dessa forma, as palavras ou expressões com maior probabilidade, dentre as opções geradas pelo modelo, tendem a corresponder à tradução mais adequada (S, 2017).

Em termos práticos, um modelo de tradução estatística é treinado com grandes volumes de textos paralelos, contendo a língua de origem e sua tradução correspondente na língua de destino. A partir desses dados, o modelo aprende padrões de correspondência entre palavras e frases, sendo capaz de estimar a tradução mais provável com base nas ocorrências observadas durante o treinamento (Lokalise Team, 2025).

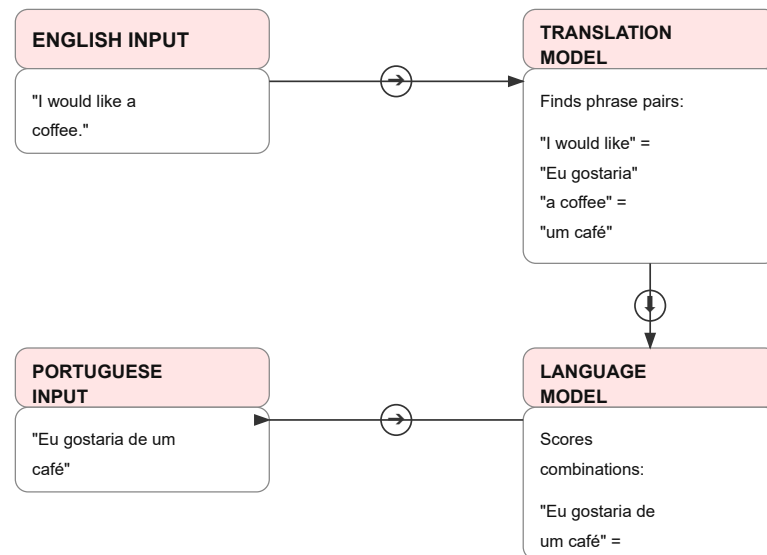
O processo de tradução estatística envolve dois módulos principais: o modelo de tradução e o modelo de linguagem (WANG et al., 2017).

O modelo de tradução tem como objetivo mapear como palavras ou frases em uma língua geralmente se correspondem em outra, realizando essa análise no nível de frases, e não apenas de palavras isoladas. Esse cuidado evita traduções que não respeitem o contexto, já que uma mesma palavra na língua de origem pode ter múltiplas traduções possíveis na língua de destino.

O modelo de linguagem, por sua vez, recebe os possíveis mapeamentos gerados pelo modelo de tradução e seleciona a frase que apresenta a maior probabilidade de ocorrência na língua de destino, considerando padrões aprendidos a partir dos textos da base de observação. Esse módulo é essencial para produzir traduções mais naturais, que se aproximem do resultado esperado de uma tradução humana.

A imagem abaixo exemplifica este processo de tradução baseado no modelo estatístico.

Figura 2 – Tradução Automática Baseada em estatística



Fonte: (Lokalise Team, 2025) adaptada para português

É notável que traduções baseadas em estatística superaram traduções baseadas em regras, em qualidade e contexto, porém, os seguintes pontos podem ser destacados sobre traduções estatísticas (WANG et al., 2017):

- Criação de base de dados complexa;
- Resultados inesperados. Fluência superficial pode ser enganosa;
- Tradução baseada em estatística não funciona bem entre línguas que possuem ordem de palavras diferentes;
- Os resultados obtidos tendem a ser mais favoráveis para línguas europeias.
- Dificuldade em lidar com dependências de longa distância em frases.

Para contornar os problemas listados acima, um novo modelo foi proposto, o qual foi a base para construção de modelos utilizados atualmente, conhecido como aprendizado profundo, em inglês, deep learning, que será abordado no próximo capítulo.

2.3 Tradução Automática Neural

A tradução automática neural, em sua essência, utiliza redes neurais artificiais (Artificial Neural Networks - ANN), cujo objetivo é simular, de forma simplificada, o funcionamento do cérebro humano. Em termos gerais, o sistema recebe uma entrada, neste caso, um texto na

língua de origem, e por meio de módulos internos da rede neural, realiza a tradução para a língua de destino (LaraTranslate, 2025).

O processo de tradução é realizado por meio de dois componentes principais: o codificador (encoder) e o decodificador (decoder) (WANG et al., 2022).

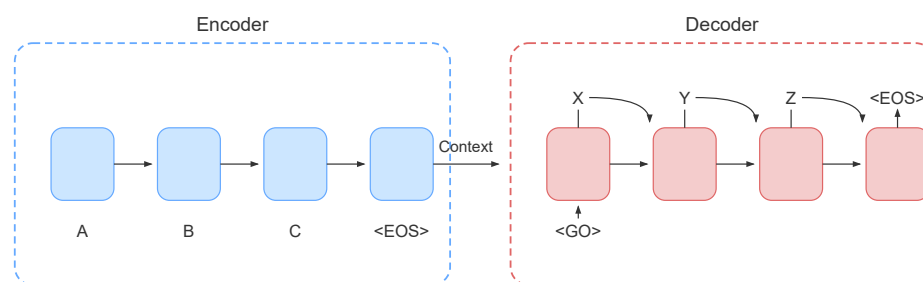
O componente de codificação tem como objetivo mapear o texto de entrada em um vetor de valores reais de múltiplas dimensões. Esse vetor representa, de forma abstrata, informações relevantes necessárias para a realização da tradução. A partir dessa representação vetorial, o componente de decodificação utiliza essas informações para gerar o texto correspondente na língua de destino.

De forma geral, o processo de tradução automática neural pode ser descrito pelas seguintes etapas (LaraTranslate, 2025):

- Conversão das palavras em vetores numéricos;
- Processamento desses vetores por meio de uma rede neural com múltiplas camadas;
- Geração de uma distribuição de probabilidade para possíveis traduções;
- Seleção da tradução mais provável.

A Figura 3 ilustra o processo de tradução utilizando redes neurais artificiais, destacando os componentes de codificação e decodificação.

Figura 3 – Tradução automática neural



Fonte: (ELAFFENDI; ALRAJHI, 2022)

O uso da tradução automática neural (Neural Machine Translation – NMT) representou, por muitos anos, uma substituição às técnicas de tradução baseadas em métodos estatísticos. Com a utilização da NMT, tornou-se possível gerar traduções sem a necessidade de intervenção humana direta, baseando-se inteiramente em grandes bases de textos e suas respectivas traduções, abordagem conhecida como aprendizado fim a fim (end-to-end) (LaraTranslate, 2025).

Essa abordagem possibilitou a geração de traduções com maior captura de contexto e maior naturalidade quando comparadas às traduções estatísticas e baseadas em regras, além de melhorar a modelagem das relações entre palavras em uma sentença, um dos principais desafios enfrentados pelas abordagens anteriores (WANG et al., 2017).

Um marco importante na adoção da tradução automática neural foi a substituição dos modelos estatísticos pelo uso de NMT no Google Tradutor, realizada pela empresa Google em 2016 (SCHUSTER; JOHNSON; THORBY, 2016), evidenciando uma mudança significativa na forma como sistemas de tradução eram desenvolvidos até então.

Apesar dos avanços proporcionados pela tradução automática neural em relação aos métodos anteriores, ainda existem limitações associadas aos modelos baseados em componentes de codificação e decodificação. Dentre essas limitações, destacam-se (LaraTranslate, 2025):

- Dificuldade na paralelização do processamento de sequências de entrada;
- Limitações no tratamento de dependências de longo alcance entre palavras;
- Maior complexidade no processo de treinamento;
- Qualidade inferior em determinados cenários de tradução.

Com o objetivo de contornar essas limitações, novas abordagens foram propostas, incluindo o uso de mecanismos de atenção, melhorias na paralelização e, posteriormente, o desenvolvimento da arquitetura Transformer.(VASWANI et al., 2017)

Dessa forma, o próximo capítulo tem como objetivo apresentar essas abordagens, bem como conceitos relacionados que contribuem para o entendimento da evolução dos modelos de tradução automática neural até os modelos mais recentes, que apresentam elevado desempenho na tarefa de tradução entre línguas.

3 Referencial teórico

3.1 Tradução Automática Neural - NMT

Como apresentado na Seção 2.3, a tradução automática neural é uma importante tarefa que tem como objetivo realizar a tradução de uma língua de origem para uma língua de destino por meio de sistemas computacionais, mais especificamente utilizando redes neurais artificiais. Essas redes buscam simular, de forma simplificada, o funcionamento do cérebro humano, permitindo a geração de traduções que se aproximem, o máximo possível, de traduções realizadas por humanos (TAN et al., 2020).

As seções seguintes têm como objetivo apresentar os conceitos teóricos que fundamentam o estudo da arquitetura Transformer, proposta com o intuito de aprimorar a tarefa de tradução automática. Essa arquitetura introduz novos componentes e combina técnicas já existentes de forma mais eficiente. Como resultado, a arquitetura Transformer estabeleceu um novo estado da arte na tradução automática baseada em redes neurais artificiais (VASWANI et al., 2017).

3.2 Tokenização

Tokenização, ou *tokenization* em inglês, é uma etapa de pré-processamento fundamental no processamento de linguagem natural. Esse processo consiste em converter textos em unidades menores, como bytes, caracteres, subpalavras, palavras ou expressões de múltiplas palavras, denominadas *tokens*.

A Figura abaixo ilustra esse processo, no qual um texto é segmentado em tokens e, posteriormente, cada token é representado por um número, comumente chamado de identificador ou id, cujo objetivo é mapear um token de volta à uma palavra ao final de uma tradução entre línguas (ASGARI; KHEIR; JAVAHERI, 2025).

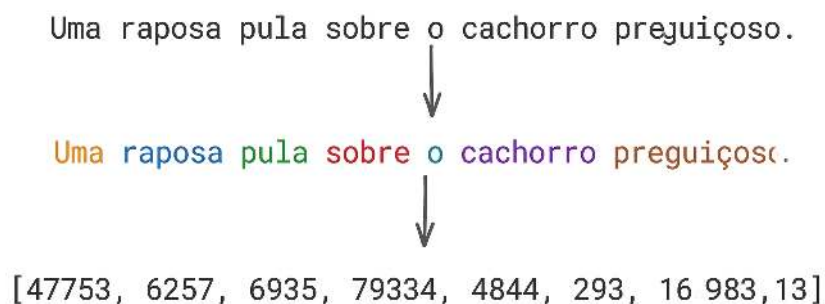


Figura 4 – Tokenização de uma frase em português

Fonte: Criada pelo autor baseado na ferramenta openai tokenizer

3.3 Embedding

O processamento de linguagem natural, em conjunto com tarefas de tradução entre línguas, exige que palavras ou tokens sejam representados de forma numérica. Nesse contexto, *embeddings* correspondem ao processo de representação de palavras ou tokens em um espaço vetorial de múltiplas dimensões, no qual relações semânticas podem ser modeladas por meio de relações matemáticas.

A representação de tokens em vetores de múltiplas dimensões é fundamental, pois esses vetores são capazes de capturar informações semânticas relevantes. Dessa forma, palavras com significados semelhantes tendem a possuir representações vetoriais próximas nesse espaço. Por exemplo, os vetores associados às palavras “gato” e “cachorro” tendem a ser mais próximos entre si do que em relação à palavra “morango”.

Essa propriedade é essencial para a geração de traduções mais naturais e coerentes, uma vez que permite aos modelos capturar relações semânticas entre palavras e contextos linguísticos (HESSANI, 2025).

A Figura abaixo ilustra o processo de conversão de uma sequência de tokens em representações vetoriais em um espaço de multiplas dimensões.

id do token	token	Vetor de Embedding (1536 dimensões)
47753	Uma	[-0.027466, 0.002899, -0.005188 ... 0.021606]
6257	raposa	[-0.018555, 0.000912, 0.010986 ... -0.015991]
6935	pula	[-0.026978, -0.012939, 0.021362 ... 0.042725]
79334	sobre	[-0.012085, 0.001244, -0.069336 ... -0.001213]
4844	o	[-0.001785, -0.008789, 0.006195 ... -0.016235]
293	cachorro	[0.016357, -0.039062, 0.045898 ... 0.001686]
168983	preguiçoso	[-0.000721, -0.021118, 0.027710 ... -0.051270]
13	.	[-0.047852, 0.057861, -0.069336 ... 0.005280]

Figura 5 – Conversão de tokens em embeddings

Fonte: (HESSANI, 2025) adaptada pelo autor

3.4 Função Softmax

Softmax é uma função matemática frequentemente utilizada como função de ligação probabilística para dados categóricos não ordenados. Dado um vetor de entradas, a aplicação da função softmax resulta em uma saída composta por uma distribuição de probabilidades, na qual cada valor está associado a um elemento da entrada.

A função softmax é amplamente utilizada em modelos de aprendizado de máquina, especialmente em tarefas que envolvem classificação e tradução automática, sendo respon-

sável por converter pontuações geradas pelo modelo em distribuições de probabilidade sobre os possíveis resultados de saída.

A função softmax é definida matematicamente como:

$$p_i = \frac{\exp(\alpha s_i)}{\sum_j \exp(\alpha s_j)} \quad (3.1)$$

onde:

- α : parâmetro de escala da função softmax, frequentemente omitido e considerado igual a 1;
- s_i : valor da i-ésima posição do vetor de pontuações.

A Figura 6 ilustra o funcionamento da função softmax aplicada a um vetor de pontuações, resultando na geração de uma distribuição de probabilidades associada a cada elemento de entrada.

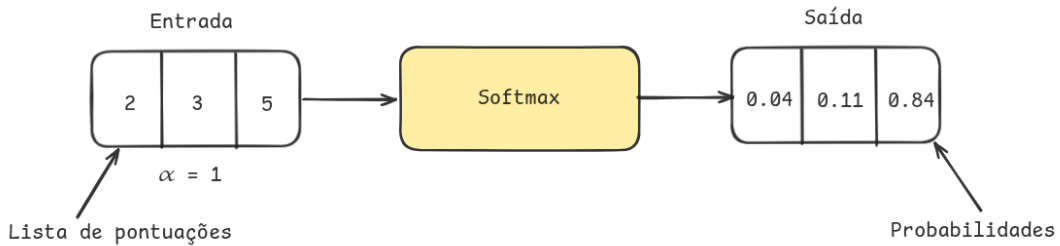


Figura 6 – Conversão de pontuações em probabilidades

Fonte: (FRANKE; DEGEN, 2023) adaptada pelo autor

Uma das propriedades mais importantes da função softmax em modelos neurais é a sua capacidade de normalização, garantindo que a soma das probabilidades geradas seja igual a 1. Essa característica permite interpretar a saída do modelo como uma distribuição de probabilidade válida sobre os possíveis resultados. A Figura 7 ilustra essa propriedade (FRANKE; DEGEN, 2023).

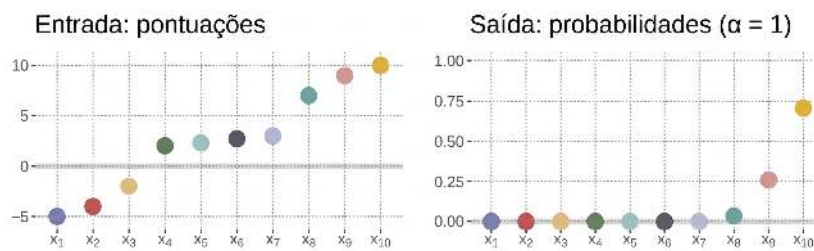


Figura 7 – Propriedade de normalização da função softmax

Fonte: (FRANKE; DEGEN, 2023)

3.5 Mecanismo De Atenção

O mecanismo de atenção é uma técnica cujo objetivo é identificar e relacionar as partes mais relevantes dos dados de entrada durante o processamento de sequências.

Grande parte das arquiteturas de tradução automática baseadas em redes neurais utiliza dois componentes principais: o codificador (encoder) e o decodificador (decoder), conforme apresentado na Seção 2.3. No entanto, modelos tradicionais baseados apenas nessa estrutura enfrentam limitações ao representar sequências longas, uma vez que toda a informação precisa ser comprimida em um vetor de dimensão fixa.

Nesse contexto, os mecanismos de atenção modificam a forma como redes neurais processam dados sequenciais, permitindo acesso direto a todas as posições da sequência de entrada. Isso elimina a necessidade de compressão em uma única representação vetorial e possibilita uma melhor captura de relações contextuais.

A operação de atenção consiste na computação de uma soma ponderada sobre os dados de entrada, na qual cada peso representa a importância de uma determinada posição da sequência para o elemento que está sendo processado. Na prática, isso permite que diferentes partes da entrada se relacionem entre si, possibilitando a captura de dependências contextuais entre palavras em tarefas de tradução.

As equações a seguir definem a formulação matemática de um mecanismo de atenção (HAYS, 2026):

$$e_{ij} = \text{score}(\mathbf{q}_i, \mathbf{k}_j) \quad (3.2)$$

$$\alpha_{ij} = \text{softmax}(e_{ij}) \quad (3.3)$$

$$\mathbf{c}_i = \sum_{j=1}^n \alpha_{ij} \mathbf{v}_j \quad (3.4)$$

Onde:

- $\mathbf{q}_i$: vetor de consulta (*query*), representando a informação buscada na posição i ;
- $\mathbf{k}_j$: vetor de chave (*key*), representando a informação disponível na posição j ;
- $\mathbf{v}_j$: vetor de valor (*value*), contendo a informação a ser agregada;
- α_{ij} : peso de atenção, formando uma distribuição de probabilidade sobre as posições de entrada, indicando o grau de contribuição de cada posição para a saída na posição i .

A Figura 8 ilustra um exemplo de tradução automática sem o uso de mecanismos de atenção, no qual é possível observar limitações na captura de relações entre palavras da língua de origem e da língua de destino.

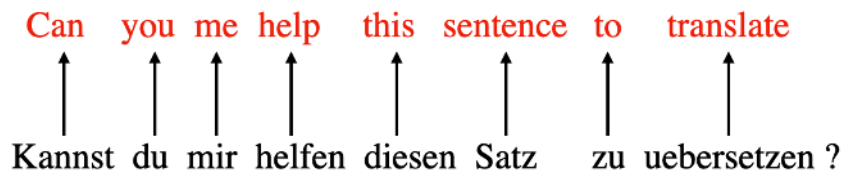


Figura 8 – Tradução sem a utilização de um mecanismo de atenção

Fonte: (RASCHKA, 2023)

Por outro lado, a Figura 9 apresenta o uso do mecanismo de atenção, evidenciando a capacidade do modelo de capturar relações entre palavras, resultando em traduções mais coerentes e contextualizadas.

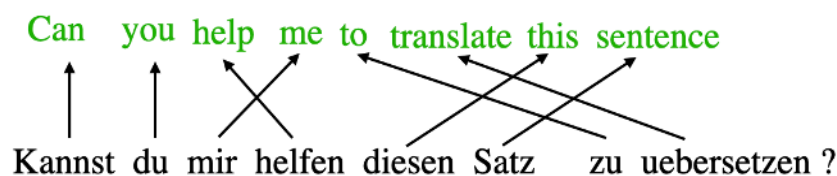


Figura 9 – Tradução com a utilização de um mecanismo de atenção

Fonte: (RASCHKA, 2023)

3.6 Arquitetura Transformer

3.6.1 Visão Geral

A arquitetura Transformer foi proposta em 2017 pela empresa Google no artigo intitulado “*Attention Is All You Need*”, representando uma mudança de paradigma em relação aos modelos até então predominantes, baseados em redes neurais recorrentes (RNN).

Essa arquitetura introduziu avanços importantes, como a capacidade de capturar dependências de longo alcance entre elementos de uma sequência e o processamento paralelo dos dados, resultando em um novo estado da arte em diversas tarefas de processamento de linguagem natural.

A partir da arquitetura Transformer, tornou-se possível o desenvolvimento de modelos avançados de geração de texto, como o GPT, da OpenAI, o Gemini, da Google, e o LLaMA, da Meta, além de impulsionar a evolução de modelos de tradução automática, como o MarianMT, de código aberto, e os modelos M2M e NLLB, também desenvolvidos pela Meta. Esses modelos representam avanços significativos na tarefa de tradução entre línguas, todos baseados nos princípios introduzidos pela arquitetura Transformer (C4LAB, 2023).

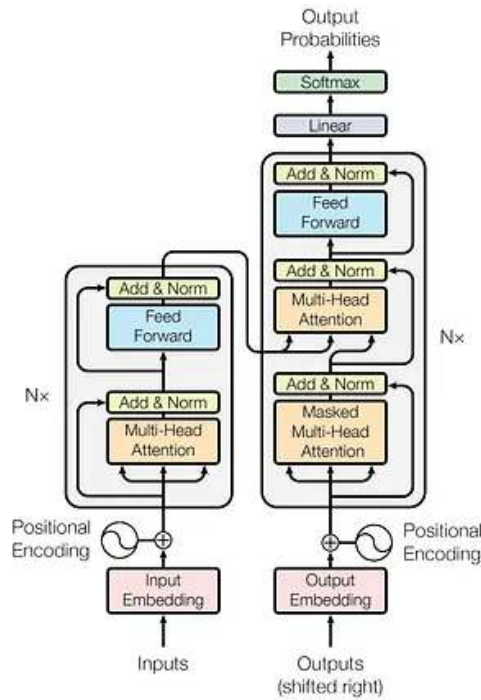


Figura 10 – Arquitetura transformer

Fonte: (VASWANI et al., 2017)

3.6.2 Positional Encoding

O *positional encoding*, ou codificação posicional, é uma técnica utilizada para incorporar informações sobre a posição dos tokens em uma sequência na arquitetura Transformer.

Em tarefas de tradução automática, a ordem das palavras desempenha um papel fundamental na construção do significado semântico de uma sentença. A posição dos termos permite identificar relações de dependência entre palavras, além de influenciar diretamente a estrutura da frase na língua de destino, uma vez que diferentes idiomas apresentam diferentes organizações sintáticas.

Como a arquitetura Transformer não possui mecanismos recorrentes ou convolucionais, ela não é capaz de capturar informações de ordem de forma implícita. Para contornar essa limitação, são adicionadas informações posicionais aos embeddings de entrada por meio de funções determinísticas baseadas em seno e cosseno (IBM, 2024a).

A codificação posicional é definida pelas seguintes equações (VASWANI et al., 2017):

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (3.5)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (3.6)$$

Onde:

- pos : posição do token na sequência;
- i : índice da dimensão do vetor de embedding;
- d_{model} : dimensão do vetor de embedding;
- PE : função de codificação posicional que associa uma posição da sequência a uma representação vetorial.

Essas funções permitem que cada posição na sequência seja representada por um vetor único, preservando relações relativas entre posições, o que auxilia o modelo na identificação da ordem dos tokens.

A Figura 11 ilustra o processo de aplicação do *positional encoding* em uma sequência de entrada.

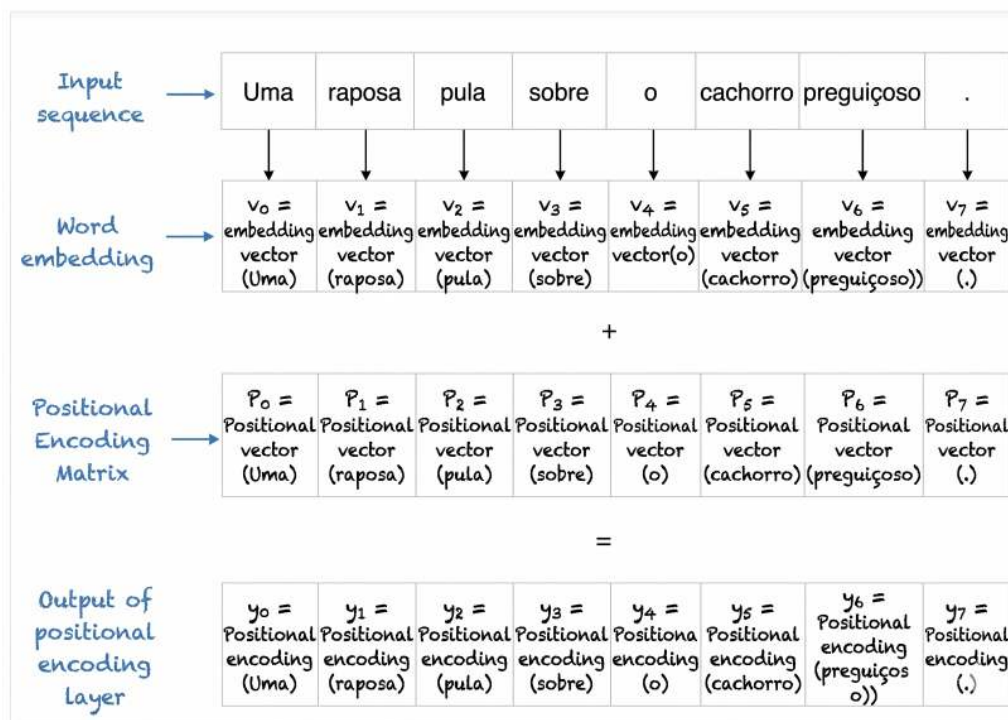


Figura 11 – Exemplo de codificação posicional aplicada a uma sequência

Fonte: (BROWNLEE, 2022) adaptado

3.6.3 Feed-Forward Neural Network

Redes neurais *feed-forward* são modelos computacionais nos quais a informação flui em uma única direção, da entrada para a saída, sem a presença de ciclos ou realimentação. Essas redes são compostas por camadas de neurônios interconectados, nas quais cada conexão possui um peso associado, responsável por ajustar a influência dos dados ao longo do processamento.

De forma geral, uma rede *feed-forward* é estruturada em três tipos de camadas: camada de entrada, responsável por receber os dados, camadas ocultas, responsáveis pelo pro-

cessamento das informações por meio de transformações não lineares, e camada de saída, que produz o resultado final do modelo.(SAZLI, 2006)

No contexto da arquitetura Transformer, as redes *feed-forward* são aplicadas de forma independente a cada posição da sequência, após o mecanismo de atenção. Seu principal objetivo é transformar e refinar as representações vetoriais geradas pelas camadas de atenção, contribuindo para a modelagem de relações mais complexas entre os elementos da sequência.(C4Lab, 2024)

A Figura 12 ilustra um exemplo de rede neural *feed-forward* com uma camada oculta.

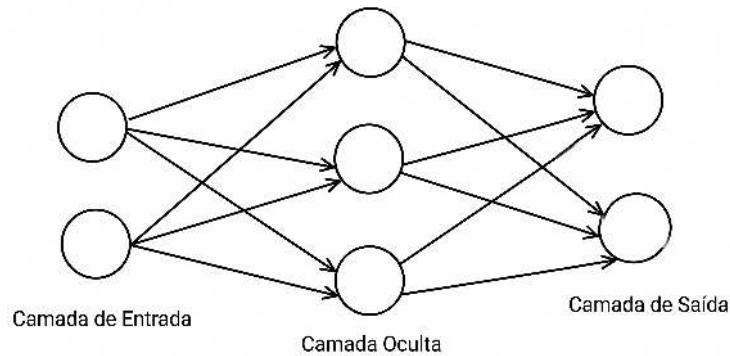


Figura 12 – Rede neural feed-forward

Fonte: (GeeksforGeeks, 2024) adaptado

3.6.4 Scaled Dot-Product Attention

No contexto da arquitetura Transformer, o mecanismo de atenção é implementado por meio da chamada *Scaled Dot-Product Attention*. Esse mecanismo calcula a relevância entre diferentes elementos da sequência de entrada utilizando três componentes principais: consultas (*queries*), chaves (*keys*) e valores (*values*).

A atenção é calculada por meio do produto escalar entre consultas e chaves, seguido de uma normalização utilizando a função softmax, conforme a equação (VASWANI et al., 2017):

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3.7)$$

onde:

- Q : Representa a perspectiva da palavra atual. Para uma palavra específica i , seu vetor de consulta(Q) é utilizado para comparar com todas as palavras disponíveis;
- K : Representa o rótulo ou identificador da informação que uma palavra carrega. Cada palavra j possui um vetor de chaves(K), onde cada consulta Q_i é comparada com todas as chaves, K_i , com o objetivo de calcular a atenção entre as palavras, indicando o quão relevante a palavra j é para a palavra i ;

- V : Representa o conteúdo atual ou significado de uma palavra atual. Onde cada palavra j possui também um vetor de valores v_j . Uma vez que a atenção entre Q_i e K_j é computado, este valor é utilizado para ajustar a relevância do vetor de valores correspondentes V_j (IBM, 2024b).

O fator de escala $\sqrt{d_k}$ é utilizado para evitar valores muito elevados no produto escalar, o que poderia prejudicar a estabilidade da função softmax.

3.6.5 Multi-Head Attention

O mecanismo de *multi-head attention* é uma extensão do mecanismo de atenção utilizado na arquitetura Transformer, cujo objetivo é permitir que o modelo capture diferentes tipos de relações entre os elementos de uma sequência de forma paralela.

Diferentemente da atenção simples, que realiza uma única projeção sobre as matrizes de consultas, chaves e valores, o *multi-head attention* projeta essas matrizes múltiplas vezes em diferentes subespaços vetoriais. Cada uma dessas projeções é processada por um mecanismo de atenção independente, denominado *head*.

Essa abordagem permite que o modelo aprenda diferentes representações e relações entre os tokens, como dependências sintáticas e semânticas, contribuindo para uma compreensão mais rica do contexto da sequência (LIU; LIU; HAN, 2021).

A formulação matemática do mecanismo de *multi-head attention* é dada por (VASWANI et al., 2017):

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O \quad (3.8)$$

onde cada *head* é definido como:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (3.9)$$

Nesse contexto, W_i^Q , W_i^K e W_i^V são matrizes de projeção aprendidas para cada *head* de atenção, enquanto W^O é uma matriz responsável por combinar as saídas concatenadas das diferentes *heads* (CORDONNIER; LOUKAS; JAGGI, 2020).

3.6.6 Encoder

Encoder, ou codificador, é um dos principais componentes da arquitetura Transformer, responsável por receber a sequência de entrada já convertida em embeddings e enriquecida com informações de posição por meio do *positional encoding*. Seu objetivo é transformar essa sequência em um conjunto de representações vetoriais contextuais, capazes de capturar relações semânticas relevantes entre os tokens.

Cada bloco do encoder é composto por duas subcamadas principais. A primeira é o mecanismo de atenção (*self-attention*), responsável por relacionar os tokens da sequência entre si, permitindo a captura de dependências de curto e longo alcance. A segunda subcamada

é a rede *feed-forward*, aplicada de forma independente a cada posição da sequência, com o objetivo de transformar e enriquecer as representações vetoriais geradas pela etapa anterior.

Após cada subcamada, são aplicadas operações de normalização e conexões residuais, com a finalidade de estabilizar o treinamento e preservar informações importantes ao longo das camadas do modelo.

Ao final do processamento pelo encoder, é gerado um conjunto de vetores de representações contextuais, no qual cada vetor está associado a um token da sequência de entrada. Esses vetores incorporam informações sobre o contexto global da sentença, incluindo relações entre palavras, e são posteriormente utilizados pelo decoder para a geração da tradução na língua de destino (C4Lab, 2024).

3.6.7 Decoder

Decoder, ou decodificador, é o componente da arquitetura Transformer responsável por gerar a sequência de saída, ou seja, a tradução na língua de destino, com base nas representações contextuais produzidas pelo encoder.

Assim como o encoder, o decoder é composto por múltiplos blocos empilhados, sendo cada bloco formado por três subcamadas principais: *Masked Self-Attention*, *Cross-Attention* (atenção sobre o encoder) e uma rede *feed-forward*.

A subcamada de *Masked Self-Attention* aplica o mecanismo de atenção sobre a própria sequência de saída, porém com uma máscara que impede o modelo de acessar tokens futuros. Dessa forma, a geração ocorre de maneira autoregressiva, considerando apenas os tokens já produzidos até o momento, o que garante coerência na construção da tradução.

A subcamada de *Cross-Attention* permite que o decoder utilize as informações produzidas pelo encoder, estabelecendo relações entre os tokens da sequência de saída e os tokens da sequência de entrada. Esse mecanismo é essencial para alinhar corretamente a tradução com o contexto da frase original.

Além disso, o mecanismo de atenção utilizado é do tipo *multi-head attention*, que realiza múltiplas operações de atenção em paralelo, permitindo ao modelo capturar diferentes tipos de relações entre os elementos da sequência.

Por fim, a camada *feed-forward* é aplicada com o objetivo de transformar e refinar as representações geradas, contribuindo para a produção de uma saída mais consistente e contextualizada.

Durante o processo de geração, o decoder produz a tradução de forma iterativa, gerando um token por vez. A cada etapa, o token gerado é realimentado como entrada para o próprio decoder, juntamente com as representações do encoder, até que um token especial de término seja produzido, indicando o fim da tradução (C4Lab, 2024).

3.7 Modelos de Tradução Open Source

3.7.1 MarianMT

MarianMT é um *framework* de tradução automática neural desenvolvido por pesquisadores da Adam Mickiewicz University, em Poznań, e da University of Edinburgh. Implementado majoritariamente na linguagem C++, o MarianMT foi projetado com foco em eficiência computacional e baixo número de dependências externas.

Diferentemente de modelos prontos para uso, o MarianMT consiste em uma plataforma para desenvolvimento, treinamento e implementação de modelos de tradução automática neural. Ele oferece suporte à construção de modelos baseados em diferentes arquiteturas, incluindo redes neurais recorrentes e, principalmente, modelos baseados na arquitetura Transformer.

Uma das principais características do MarianMT é sua otimização para desempenho, permitindo o treinamento eficiente em múltiplas unidades de processamento, como CPUs e GPUs. Além disso, o framework possibilita a criação de modelos especializados para diferentes pares de línguas, a partir de bases de dados específicas (JUNCZYS-DOWMUNT et al., 2018).

Entre os principais aspectos do MarianMT, destacam-se (JUNCZYS-DOWMUNT et al., 2018):

- Implementação eficiente em C++, com foco em desempenho;
- Suporte a treinamento e inferência em CPU e GPU;
- Compatibilidade com arquiteturas modernas de tradução neural, como Transformer;
- Código aberto sob licença permissiva (MIT).

3.7.2 M2M-100

M2M-100 (*Many-to-Many*) é um modelo de tradução automática neural baseado na arquitetura transformer e desenvolvido pela Meta, cujo objetivo é realizar traduções diretas entre múltiplos pares de línguas, mais precisamente entre 100 línguas, sem a necessidade de utilizar o inglês como língua intermediária.

Em abordagens tradicionais, quando a língua de origem ou destino não envolve o inglês, é comum a utilização de uma estratégia intermediária, na qual a tradução é realizada inicialmente para o inglês e, posteriormente, convertida para a língua de destino. Essa abordagem é frequentemente adotada devido à maior disponibilidade de dados de treinamento em inglês.

O modelo M2M-100 foi proposto para superar essa limitação, permitindo traduções diretas entre diferentes línguas. Para isso, o modelo foi treinado em uma grande base de dados multilíngue, construída com o objetivo de abranger uma ampla variedade de pares linguísticos.

Além disso, foram utilizadas estratégias para melhorar a qualidade dos dados, incluindo o agrupamento de línguas com base em similaridades linguísticas, geográficas e culturais. Es-

sas técnicas contribuíram para a geração de traduções mais consistentes e contextualizadas, especialmente para pares de línguas com menor disponibilidade de dados.

O desenvolvimento do M2M-100 atende à necessidade de plataformas globais, como as aplicações da Meta, de oferecer suporte à comunicação entre usuários de diferentes idiomas com traduções de alta qualidade (FAN et al., 2020).

3.7.3 NLLB

NLLB (*No Language Left Behind*) é um modelo de tradução automática neural baseado na arquitetura Transformer, desenvolvido pela Meta, com o objetivo de realizar traduções de alta qualidade entre um grande número de línguas, incluindo línguas com poucos recursos disponíveis.

Diferentemente do modelo M2M-100, apresentado na Seção 3.7.2, o NLLB representa uma evolução significativa, sendo projetado para suportar mais de 200 línguas, com foco especial na melhoria da qualidade de tradução para línguas de baixo recurso (*low-resource languages*).

Um dos principais desafios enfrentados no desenvolvimento do NLLB está relacionado à escassez de dados paralelos para determinadas línguas. Para contornar esse problema, foram utilizadas estratégias de construção e curadoria de bases de dados multilíngues em larga escala, incluindo a coleta de textos de diversas fontes, como entrevistas com nativos da língua e o uso de técnicas de filtragem para garantir maior qualidade dos dados.

Além disso, o treinamento do modelo envolveu o uso de técnicas avançadas de otimização e regularização, com o objetivo de melhorar a generalização e reduzir problemas como o *overfitting*, especialmente em cenários com dados limitados (TEAM et al., 2022)

3.8 Métricas de Avaliação

3.8.1 BLEU

A métrica BLEU (*Bilingual Evaluation Understudy*) é um método automático utilizado para avaliação de traduções automáticas. Trata-se de uma métrica independente de linguagem, cujo objetivo é medir o grau de similaridade entre uma tradução gerada automaticamente e uma ou mais traduções de referência produzidas por humanos.

A ideia central da métrica BLEU é que traduções automáticas de maior qualidade tendem a apresentar maior sobreposição com traduções humanas de referência. Dessa forma, quanto maior a similaridade entre a tradução gerada e a tradução de referência, maior será o valor da métrica BLEU.

O valor da métrica BLEU varia no intervalo de 0 a 1, onde valores mais próximos de 1 indicam maior similaridade com a tradução de referência, enquanto valores próximos de 0 indicam baixa similaridade (PAPINENI et al., 2002).

A formulação matemática da métrica BLEU é definida como:

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}.$$

Então,

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right).$$

Onde:

- c : comprimento da tradução gerada;
- r : comprimento da tradução de referência;
- w_n : pesos associados a cada ordem de n-grama, tais que $\sum w_n = 1$;
- p_n : precisão modificada de n-gramas.

3.8.2 BERTScore

BERTScore é uma métrica automática utilizada para avaliação de textos gerados por modelos de processamento de linguagem natural, incluindo tarefas de tradução automática.

Diferentemente de métricas baseadas em correspondência exata de n-gramas, como o BLEU, o BERTScore avalia a similaridade entre uma sentença gerada e uma sentença de referência por meio de representações vetoriais contextuais (*embeddings*) obtidas a partir de modelos pré-treinados, como o BERT.

O cálculo do BERTScore é realizado por meio da comparação entre os embeddings de cada token da sentença gerada e os embeddings dos tokens da sentença de referência. Para cada token, é calculada uma medida de similaridade (geralmente baseada em cosseno), permitindo capturar relações semânticas mesmo quando não há correspondência exata entre as palavras.

A avaliação por meio do BERTScore é composta pelas métricas de precisão, revocação e F1-score.

Onde:

- Precisão: considera a sentença gerada pelo modelo e mede o quanto os tokens gerados estão semanticamente alinhados com a sentença de referência;
- Revocação: considera a sentença de referência e avalia o quanto seu conteúdo está presente na sentença gerada pelo modelo;
- F1-score: combina precisão e revocação em um único valor, representando um equilíbrio entre a cobertura do conteúdo da referência e a qualidade da correspondência semântica.

O valor do F1-score é frequentemente utilizado como resultado principal da métrica BERTScore, sendo que valores mais elevados indicam maior similaridade semântica entre a sentença gerada e a sentença de referência (ZHANG et al., 2019).

3.8.3 chrF

ChrF (*CHaracter n-gram F-score*) é uma técnica utilizada para avaliação automática de modelos de tradução, baseada na comparação de n-gramas de caracteres entre a sentença gerada e a sentença de referência (POPOVIĆ, 2015).

Diferentemente de métricas como o BLEU, que operam no nível de palavras, o chrF realiza a comparação no nível de caracteres, permitindo maior sensibilidade a variações morfológicas e flexionais presentes nas línguas naturais.

O cálculo da métrica chrF baseia-se na combinação de precisão e revocação de n-gramas de caracteres, resultando em uma medida do tipo F-score. Essa abordagem permite capturar similaridades entre palavras mesmo quando há pequenas variações, como flexões ou erros parciais na tradução (MIND, 2026).

A formulação matemática da métrica chrF é definida como (POPOVIĆ, 2015):

$$\text{CHRF}_\beta = (1 + \beta^2) \frac{\text{CHRP} \cdot \text{CHRR}}{\beta^2 \cdot \text{CHRP} + \text{CHRR}} \quad (3.10)$$

onde:

- chrP: precisão de n-gramas de caracteres, representando a proporção de n-gramas presentes na sentença gerada que também aparecem na referência;
- chrR: revocação de n-gramas de caracteres, representando a proporção de n-gramas da referência que são recuperados pela sentença gerada;
- β : parâmetro que controla a importância relativa entre precisão e revocação. Quando $\beta = 1$, ambas possuem o mesmo peso.

3.9 Eficiência Computacional

Eficiência computacional refere-se à capacidade de um algoritmo, programa ou sistema de computador de realizar uma determinada tarefa utilizando a menor quantidade possível de recursos (DUENHA, 2024).

Entre os principais recursos utilizados na avaliação da eficiência computacional, destacam-se:

- Tempo de execução: corresponde ao tempo necessário para que um programa execute uma tarefa específica. Esse tempo pode ser interpretado como o número de operações primitivas ou passos realizados pelo algoritmo para uma determinada entrada;

- Consumo de recursos computacionais: envolve a quantidade de memória RAM, uso de CPU e armazenamento em disco necessários para a execução do algoritmo ou sistema;
- Análise assintótica: geralmente expressa por meio da notação Big O, descreve como o tempo de execução ou o uso de recursos cresce em função do tamanho da entrada, especialmente quando esse tamanho tende ao infinito.

4 Procedimentos metodológicos

Segundo (GIL et al., 2002), a pesquisa experimental consiste na definição de um objeto de estudo, na seleção de variáveis capazes de influenciá-lo e na determinação de formas de controle e observação dos efeitos produzidos por essas variáveis. Com base nessa definição, o presente trabalho caracteriza-se como uma pesquisa de natureza experimental, uma vez que modelos de tradução automática neural baseados na arquitetura Transformer serão comparados e submetidos a diferentes cenários e bases de dados.

Além disso, de acordo com (FONSECA, 2002), a pesquisa quantitativa utiliza linguagem matemática para descrever fenômenos e analisar relações entre variáveis. Nesse contexto, este trabalho também possui caráter quantitativo, pois os modelos serão avaliados por meio de métricas como BLEU, BERTScore e chrF, bem como por medidas de eficiência computacional, obtidas a partir de experimentos controlados.

4.1 Modelos

Nesta seção são descritas as principais características dos modelos selecionados para comparação na tarefa de tradução automática, incluindo informações como tamanho, finalidade e forma de acesso.

A seguir, são apresentados os modelos utilizados neste trabalho, bem como suas respectivas configurações e locais para obtenção:

- **MarianMT**: O modelo utilizado será o *opus-mt-tc-big-en-pt*, com aproximadamente 0,2 bilhões de parâmetros. Trata-se de um modelo específico para tradução do inglês para o português. O seguinte link direciona para a plataforma *Hugging Face*, onde o modelo pode ser obtido para uso local.
- **M2M-100**: O modelo utilizado será o *m2m100_418M*, com aproximadamente 0,4 bilhões de parâmetros. Diferentemente do MarianMT, este modelo não requer a seleção de um par específico de línguas, pois foi projetado para tradução direta entre múltiplos idiomas. O seguinte link redireciona para a plataforma *Hugging Face*, onde o modelo pode ser obtido para uso local.
- **NLLB**: O modelo utilizado será o *nllb-200-distilled-600M*, com aproximadamente 0,6 bilhões de parâmetros. Assim como o M2M-100, este modelo permite tradução direta entre múltiplas línguas, sem a necessidade de uma língua intermediária. O seguinte link redireciona para a plataforma *Hugging Face*, onde o modelo pode ser obtido para uso local.

4.2 Tipos de texto

Nesta seção são definidos os tipos de texto utilizados na avaliação dos modelos de tradução automática.

Os textos foram organizados em quatro categorias, considerando o tamanho e o domínio do conteúdo, conforme descrito a seguir:

- **Textos curtos:** textos com até 15 palavras de domínio conversacional e informal;
- **Textos longos:** textos com mais de 15 palavras de domínio conversacional e informal;
- **Textos específicos:** textos associados a domínios especializados, como medicina, engenharia, computação e nutrição, também obtidos a partir de bases de dados públicas.

4.3 Métricas de Avaliação

Nesta seção tem como objetivo apresentar as ferramentas utilizadas para o cálculo das métricas BLEU, BERTScore e chrF e avaliação das traduções geradas pelos modelos.

4.3.1 BLEU

O cálculo da métrica BLEU, após a geração da tradução pelo modelo, será realizado utilizando a biblioteca da linguagem Python denominada NLTK (*Natural Language Toolkit*). Essa biblioteca disponibiliza diversas funcionalidades voltadas ao processamento de linguagem natural, incluindo a função *sentence_bleu*, utilizada para calcular o valor da métrica BLEU a nível de sentença.

Essa função recebe como entrada uma ou mais sentenças de referência e a sentença gerada pelo modelo, retornando um valor numérico que representa o grau de similaridade entre elas com base em n-gramas.

Além disso, será utilizado um método de suavização (*smoothing*) para evitar valores nulos em casos de baixa sobreposição entre os n-gramas da sentença gerada e da referência.

Abaixo é apresentado um exemplo de cálculo da métrica BLEU utilizando a função *sentence_bleu* da biblioteca NLTK:


```
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction

referencia = [['Uma', 'raposa', 'pula', 'sobre', 'o', 'cachorro', '.']]
geradoPeloModelo = ['Uma', 'pula', 'raposa', 'sobre', 'o', 'cachorro', ',']

smooth = SmoothingFunction().method1

print(sentence_bleu(referencia, geradoPeloModelo, smoothing_function=smooth))
```

Listing 4.1 – Exemplo de cálculo da métrica BLEU com NLTK

4.3.2 BERTScore

O cálculo da métrica BERTScore será realizado utilizando a biblioteca da linguagem Python denominada *bert-score*. Essa biblioteca permite calcular a similaridade entre sentenças por meio da comparação de *embeddings* contextuais gerados a partir de modelos pré-treinados.

A função *score* é responsável por receber como entrada a sentença gerada pelo modelo e a sentença de referência, retornando três medidas principais: precisão, revocação e F1-score. Para este trabalho, será utilizado o F1-score médio como medida final do BERTScore, por representar um equilíbrio entre precisão e revocação.

Abaixo é apresentado um exemplo de cálculo da métrica BERTScore utilizando a função *score* da biblioteca *bert-score*:

```
from bert_score import score

geradoPeloModelo = ["O tempo esta congelante hoje."]
referencia = ["Hoje esta frio."]

PRECISAO, REVOCAAO, F1_SCORE = score(geradoPeloModelo, referencia, lang="pt",
                                     verbose=True)

print(F1_SCORE.mean())
```

Listing 4.2 – Exemplo de cálculo da métrica BERTScore com a biblioteca bert-score

4.3.3 chrF

Para o cálculo da métrica chrF será realizada utilizando a biblioteca da linguagem python (*sacrebleu*). Essa biblioteca possui funções que realizam o cálculo de métricas para avaliação de traduções automáticas, e dentre elas a função (*sentence_chrf*), cujo objetivo é realizar o cálculo da métrica chrF dado a sequência gerada pelo modelo e a sequência de referência.

A função (*sentence_chrf*) recebe dois argumentos, o primeiro sendo a sequência gerada pelo modelo, já o segundo argumento, a sequência de referência. Como resposta, a função retorna um valor entre 0 e 100, indicando o valor da métrica chrF

Abaixo é apresentado um exemplo de cálculo da métrica chrF utilizando a função *sentence_chrf* da biblioteca *sacrebleu*:

```
import sacrebleu

geradoPeloModelo = "The cat is on the mat."
referencia = ["The cat is on the mat."]

chrf = sacrebleu.sentence_chrf(geradoPeloModelo, referencia)

print(chrf)
```

Listing 4.3 – Exemplo de cálculo da métrica chrF com a biblioteca sacrebleu

4.4 Ambiente Computacional

Com o objetivo de garantir um ambiente controlado e uniforme para a comparação entre os modelos de tradução, optou-se pela utilização de um ambiente virtual em nuvem (*cloud*) para execução dos experimentos. A plataforma escolhida foi a **DigitalOcean**, devido à disponibilidade de instâncias com cobrança por hora e oferta de máquinas com alto desempenho, o que possibilita maior flexibilidade na execução dos testes.

A Tabela abaixo apresenta as especificações da máquina virtual utilizada nos experimentos:

Memória RAM	Núcleos (vCPU)	Transferência de dados	Armazenamento
16 GB	8 vCPUs	6 TB	100 GB

Tabela 1 – Especificações da máquina virtual utilizada

Cabe destacar que, por se tratar de um ambiente virtualizado, informações detalhadas sobre o hardware físico, como modelo do processador ou tipo de memória, não são explicitamente fornecidas pela plataforma. Isso ocorre porque a instância virtual pode ser alocada dinamicamente sobre diferentes máquinas físicas, dependendo da disponibilidade no momento da criação.

4.5 Desempenho Computacional

Nesta seção são descritas as métricas e ferramentas utilizadas para avaliação da eficiência computacional dos modelos de tradução, considerando principalmente o uso de CPU, tempo de execução e consumo de memória RAM.

4.5.1 Uso de CPU e Tempo de Execução

Neste experimento, serão considerados dois indicadores principais relacionados à CPU: o tempo médio de execução e o uso médio de CPU durante a tarefa de tradução.

Para a medição do tempo de execução, será utilizada a ferramenta de linha de comando *hyperfine*. Essa ferramenta é amplamente utilizada para a realização de benchmarks, permitindo medir o tempo total, média e desvio padrão de execução de um programa ao longo de múltiplas execuções.

Abaixo é apresentado um exemplo de uso da ferramenta *hyperfine*:

```
hyperfine 'python3 script.py'
```

Listing 4.4 – Exemplo de uso da ferramenta *hyperfine*

Para a medição do uso médio de CPU durante a execução dos modelos, será utilizada a ferramenta *pidstat*, nativa de sistemas operacionais Linux. Essa ferramenta permite coletar estatísticas detalhadas sobre o uso de recursos por processos individuais.

A seguir, um exemplo de utilização da ferramenta *pidstat*:

```
pidstat -u -e python3 test.py
```

Listing 4.5 – Exemplo de uso da ferramenta *pidstat*

As principais flags utilizadas são:

- **-u**: exibe estatísticas de utilização da CPU;
- **-e**: executa o comando especificado e monitora seu consumo de recursos.

4.5.2 Consumo de Memória RAM

Para a análise de memória RAM, serão considerados dois indicadores: o uso médio e o uso máximo (pico) durante a execução da tarefa de tradução. O uso médio de memória será obtido por meio da ferramenta *pidstat*, utilizando a flag *-r*, que permite monitorar o consumo de memória ao longo da execução do processo.

Já o uso máximo de memória (pico) será medido utilizando a ferramenta */usr/bin/time*, também disponível em sistemas Linux. Essa ferramenta fornece informações detalhadas sobre a execução de programas, incluindo o consumo máximo de memória.

Abaixo é apresentado um exemplo de uso da ferramenta *time*:

```
/usr/bin/time -v python3 script.py
```

Listing 4.6 – Exemplo de uso da ferramenta *time*

A flag *-v* permite a exibição de informações detalhadas, incluindo o valor de memória máxima utilizada durante a execução do programa.

4.6 Base de Dados

Esta seção apresenta as bases de dados utilizadas nos experimentos de avaliação dos modelos de tradução automática.

4.6.1 Textos Conversacionais e Informais

Para a composição da base de textos conversacionais e informais, será utilizado o repositório OPUS (*Open Parallel Corpus*), mais especificamente o conjunto de dados *Open-Subtitles v2024*, contendo pares de tradução entre inglês e português brasileiro.

Essa base é composta majoritariamente por diálogos extraídos de legendas de filmes e séries, sendo adequada para representar linguagem informal e conversacional.

A base de dados pode ser acessada por meio do seguinte link.

A Tabela 2 apresenta exemplos de pares de frases presentes na base de dados.

Frase em inglês	Tradução para português brasileiro
Did you do this, Dr. Ivins?	Você fez isso, Dr. Ivins?
Hello, mother. Hello, Bruce.	Olá, mãe. Olá, Bruce.
Hello, papa. What are you doing?	Olá, papai. O que você está fazendo?

Tabela 2 – Exemplo de pares de tradução da base OpenSubtitles v2024

4.6.2 Textos Específicos

Para a avaliação em textos de domínio específico, serão utilizadas bases de dados do repositório OPUS com conteúdos técnicos. Foram selecionados os conjuntos *MDN Web Docs*, referente à documentação de tecnologias web mantida pela organização Mozilla, e *PHP*, contendo documentação da linguagem de programação PHP.

As bases podem ser acessadas por meio do seguinte link.

A Tabela 3 apresenta exemplos de pares de frases da base *MDN Web Docs*:

Frase em inglês	Tradução para português brasileiro
Present, accept, interpret, calculate, repeat	Apresentar, aceitar, interpretar, calcular, repetir
The specifics depend on the game.	As especificidades dependem do jogo.
Some games drive this cycle by user input.	Alguns jogos guiam este ciclo pela entrada do usuário.

Tabela 3 – Exemplo de pares de tradução da base MDN Web Docs

A Tabela 5 apresenta exemplos de pares de frases da base *PHP*:

Frase em inglês	Tradução para português brasileiro
The documentation is stored in the phpdoc module.	A documentação é armazenada no módulo phpdoc.
How to help improve the documentation	Como ajudar a melhorar a documentação
Classify the bug as "Documentation Problem".	Classifique o problema como "Documentation Problem".

Tabela 4 – Exemplo de pares de tradução da base PHP

4.7 Procedimento de Execução

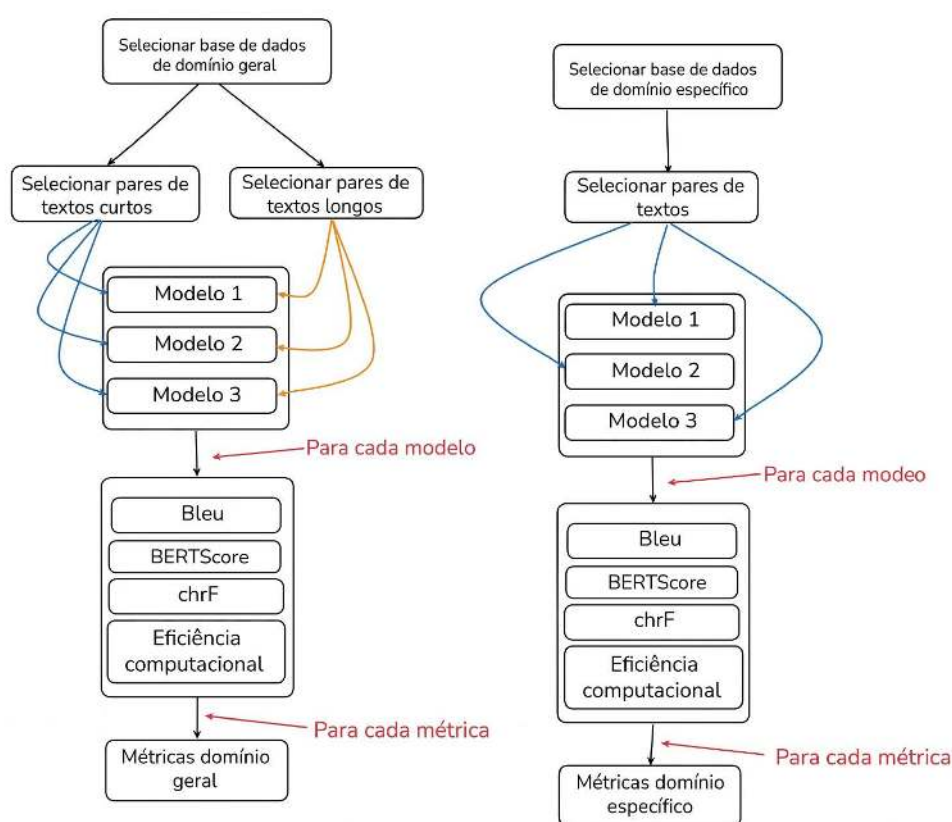


Figura 13 – Procedimento

Fonte: autor

4.8 Análise dos dados

Esta seção tem como objetivo descrever os procedimentos adotados para a análise dos dados coletados a partir dos experimentos de tradução automática. A análise considera tanto a qualidade das traduções geradas quanto o desempenho computacional dos modelos avaliados.

4.8.1 Comparação entre modelos

A comparação entre os modelos de tradução será realizada a partir da utilização de um mesmo conjunto de entradas, composto por pares de sentenças paralelas, sendo o inglês a língua de origem e o português brasileiro a língua de destino. Cada modelo receberá exatamente as mesmas sentenças de entrada, a fim de garantir a comparabilidade dos resultados.

Para cada sentença de entrada, será gerada uma tradução por cada modelo avaliado. As saídas produzidas serão então comparadas com as respectivas traduções de referência por meio das métricas BLEU, BERTScore e chrF. Além disso, serão coletadas métricas relacionadas ao desempenho computacional, incluindo uso de memória RAM, utilização de CPU e tempo de execução durante o processo de inferência.

4.8.2 Comparação entre tipos de texto

A análise também considera o impacto das características dos textos de entrada no desempenho dos modelos. Para isso, os dados serão organizados em duas dimensões principais: tamanho do texto e domínio do conteúdo.

Em relação ao tamanho, as sentenças serão classificadas em categorias previamente definidas, como textos curtos e textos longos, com base no número de palavras ou tokens. Já em relação ao domínio, os textos serão agrupados em categorias distintas, como textos conversacionais e textos técnicos, sendo estes últimos compostos por conteúdos relacionados à linguagem de programação PHP e à documentação web da fundação Mozilla.

Para cada categoria, um conjunto equivalente de sentenças será selecionado e submetido a todos os modelos avaliados. As métricas de qualidade (BLEU, BERTScore e chrF) e de desempenho computacional serão calculadas individualmente para cada categoria.

Os resultados serão analisados de forma comparativa, por meio de tabelas e representações gráficas, permitindo observar o comportamento dos modelos em diferentes cenários.

4.8.3 Correlação entre métricas

Além da análise individual das métricas, este trabalho também investiga a relação entre elas. As métricas BLEU, BERTScore e chrF possuem fundamentos distintos: enquanto o BLEU é baseado na sobreposição de n-gramas, o BERTScore considera similaridade semântica por meio de representações vetoriais, e o chrF avalia correspondências em nível de caracteres. Dessa forma, é esperado que cada métrica capture diferentes aspectos da qualidade da tradução.

Para analisar a relação entre essas métricas, será realizada uma análise de correlação estatística entre seus valores, considerando os resultados obtidos para todas as sentenças e modelos avaliados. Essa análise tem como objetivo identificar o grau de concordância entre as métricas, bem como possíveis divergências em suas avaliações.

5 Cronograma

O cronograma abaixo apresenta a previsão das atividades a serem realizadas ao longo do segundo semestre de 2026 (2026/2) para o desenvolvimento da Pesquisa.

Atividades	Ago	Set	Out	Nov	Dez
Separação dos dados a serem utilizados	X				
Execução dos experimentos	X	X			
Coleta, análise e apresentação dos resultados		X	X		
Redação final do TCC 2				X	
Ajustes necessários				X	
Revisão da Redação Final				X	X
Apresentação e entrega final					X

Tabela 5 – Cronograma das atividades propostas

Referências

- ANAND, S. et al. *Why Transformers Are Replacing RNNs in NLP?* 2025. Acesso em: 31 mar. 2026. Disponível em: <<https://medium.com/@albusdor11/why-transformers-are-replacing-rnns-in-nlp-1c1ef7ad4993>>. Citado na página 7.
- ASGARI, E.; KHEIR, Y. E.; JAVAHERI, M. A. S. *MorphBPE: A Morpho-Aware Tokenizer Bridging Linguistic Complexity for Efficient LLM Training Across Morphologies*. 2025. Disponível em: <<https://arxiv.org/abs/2502.00894>>. Citado na página 15.
- BROWNLEE, J. *A Gentle Introduction to Positional Encoding in Transformer Models, Part 1*. Machine Learning Mastery, 2022. Acessado em: 18 de abril de 2026. Disponível em: <<https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/>>. Citado na página 21.
- BULLA, O. *Exclusivo: 7 em cada 10 brasileiros compram em sites estrangeiros*. 2024. Acesso em: 31 mar. 2026. Disponível em: <<https://forbes.com.br/forbes-money/2024/08/exclusivo-7-em-cada-10-brasileiros-compram-em-sites-estrangeiros/>>. Citado na página 7.
- C4LAB. *Arquitetura Transformer: Explicação*. 2023. Acessado em: 12 de abril de 2026. Disponível em: <<https://c4lab.com.br/arquitetura-transformer-explicacao/>>. Citado na página 19.
- C4Lab. *Arquitetura Transformer: Uma explicação completa*. 2024. Acessado em: 18 de abril de 2026. Disponível em: <<https://c4lab.com.br/arquitetura-transformer-explicacao/>>. Citado nas páginas 22 e 24.
- CHAROENPORNSAWAT, P.; SORNLERLTHAMVANICH, V.; CHAROENPORN, T. Improving translation quality of rule-based machine translation. In: *COLING-02: machine translation in Asia*. [S.l.: s.n.], 2002. Citado na página 10.
- CLARO, F. D. O avanço tecnológico no mundo econômico. *Revista FAE, Vitrine da Conjuntura*, v. 2, n. 8, p. 1–4, 2009. Citado na página 7.
- CORDONNIER, J.; LOUKAS, A.; JAGGI, M. Multi-head attention: Collaborate instead of concatenate. *CoRR*, abs/2006.16362, 2020. Disponível em: <<https://arxiv.org/abs/2006.16362>>. Citado na página 23.
- DANG, K. *Language Model History — Before and After Transformer: The AI Revolution*. 2023. Acesso em: 31 mar. 2026. Disponível em: <<https://medium.com/@kirudang/language-model-history-before-and-after-transformer-the-ai-revolution-bedc7948a130>>. Citado na página 7.
- DUENHA, L. *Apostila de Introdução à Organização e Arquitetura de Computadores*. 2024. Faculdade de Computação (FACOM) - UFMS. Acessado em: 25 abr. 2026. Disponível em: <https://www.facom.ufms.br/~lianaduenha/sites/default/files/aula02_apostila.pdf>. Citado na página 28.
- ELAFFENDI, M.; ALRAJHI, K. Beyond the transformer: a novel polynomial inherent attention (pia) model and its great impact on neural machine translation. *Computational Intelligence and Neuroscience*, Wiley Online Library, v. 2022, n. 1, p. 1912750, 2022. Citado na página 13.

- FAN, A. et al. *Introducing the First Many-to-Many Multilingual Machine Translation Model That Can Translate Between Any Pair of 100 Languages*. 2020. Meta AI Blog. Accessed: 2026-04-21. Disponível em: <<https://ai.meta.com/blog/introducing-many-to-many-multilingual-machine-translation/>>. Citado na página 26.
- FEINER, L.; LESWING, K. *The race to the 'universal translator': Inside the AI battle between Apple, Google and Meta*. 2025. Acesso em: 31 mar. 2026. Disponível em: <<https://www.cnbc.com/2025/09/12/apple-google-meta-universal-translator.html>>. Citado na página 8.
- FONSECA, J. J. S. da. *Apostila de metodologia da pesquisa científica*. [S.l.]: João José Saraiva da Fonseca, 2002. Citado na página 30.
- FRANKE, M.; DEGEN, J. The softmax function: Properties, motivation, and interpretation. *PsyArXiv DOI*, v. 10, 2023. Citado na página 17.
- GeeksforGeeks. *Feedforward Neural Network in Deep Learning*. 2024. <<https://www.geeksforgeeks.org/deep-learning/feedforward-neural-network/>>. Acedido em: 24 de maio de 2024. Citado na página 22.
- GIL, A. C. et al. *Como elaborar projetos de pesquisa*. [S.l.]: Atlas São Paulo, 2002. v. 4. Citado na página 30.
- HAYS, H. *Attention mechanisms in neural networks*. 2026. Disponível em: <<https://arxiv.org/abs/2601.03329>>. Citado na página 18.
- HESSANI, H. S. *LLM Embeddings Explained: A Visual and Intuitive Guide*. 2025. Citado na página 16.
- IBM. *What is positional encoding?* 2024. Acessado em: 18 de abril de 2026. Disponível em: <<https://www.ibm.com/think/topics/positional-encoding>>. Citado na página 20.
- IBM. *What is the attention mechanism?* 2024. <<https://www.ibm.com/think/topics/attention-mechanism>>. Acedido em: 19 de abril de 2026. Citado na página 23.
- JUNCZYS-DOWMUNT, M. et al. Marian: Fast neural machine translation in C++. In: *Proceedings of ACL 2018, System Demonstrations*. Melbourne, Australia: Association for Computational Linguistics, 2018. p. 116–121. Disponível em: <<http://www.aclweb.org/anthology/P18-4020>>. Citado na página 25.
- LaraTranslate. *Neural Machine Translation: Evolution and Impact*. 2025. Acessado em: 3 abr. 2026. Disponível em: <<https://blog.laratranslate.com/neural-machine-translation-evolution-and-impact/>>. Citado nas páginas 13 e 14.
- LIU, L.; LIU, J.; HAN, J. Multi-head or single-head? an empirical comparison for transformer training. *arXiv preprint arXiv:2106.09650*, 2021. Citado na página 23.
- Lokalise Team. *Statistical Machine Translation: What is it and How it Works*. 2025. <<https://lokalise.com/blog/statistical-machine-translation/>>. Accessed: 2026-04-02. Citado nas páginas 11 e 12.
- LOPEZ, A. Statistical machine translation. *ACM Comput. Surv.*, Association for Computing Machinery, New York, NY, USA, v. 40, n. 3, ago. 2008. ISSN 0360-0300. Disponível em: <<https://doi.org/10.1145/1380584.1380586>>. Citado na página 7.
- MIND, E. *chrF Metric: Character n-gram F-score for Machine Translation*. 2026. Accessed: 2026-04-21. Disponível em: <<https://www.emergentmind.com/topics/chrf-metric>>. Citado na página 28.

- OKPOR, M. D. Machine translation approaches: issues and challenges. *International Journal of Computer Science Issues (IJCSI)*, International Journal of Computer Science Issues (IJCSI), v. 11, n. 5, p. 159, 2014. Citado na página 11.
- OLIVEIRA, A. *Apenas 1% dos brasileiros é fluente em inglês, aponta estudo*. 2026. Acesso em: 31 mar. 2026. Disponível em: <<https://www.educamaisbrasil.com.br/educacao/noticias/apenas-1-dos-brasileiros-e-fluente-em-ingles-aponta-estudo>>. Citado na página 8.
- PAPINENI, K. et al. Bleu: a method for automatic evaluation of machine translation. In: *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. [S.l.: s.n.], 2002. p. 311–318. Citado na página 26.
- POPOVIĆ, M. chrF: character n-gram f-score for automatic mt evaluation. In: *Proceedings of the tenth workshop on statistical machine translation*. [S.l.: s.n.], 2015. p. 392–395. Citado na página 28.
- RASCHKA, S. *Understanding and Coding Self-Attention and Multi-Head Attention from Scratch*. 2023. Acessado em: 12 de abril de 2026. Disponível em: <<https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>>. Citado na página 19.
- S, S. Statistical vs rule based machine translation; A case study on indian language perspective. *CoRR*, abs/1708.04559, 2017. Disponível em: <<http://arxiv.org/abs/1708.04559>>. Citado nas páginas 10 e 11.
- SAZLI, M. H. A brief review of feed-forward neural networks. *Communications Faculty of Sciences University of Ankara Series A2-A3 Physical Sciences and Engineering*, Ankara University, v. 50, n. 01, 2006. Citado na página 22.
- SCHUSTER, M.; JOHNSON, M.; THORBY, T. *Zero-Shot Translation with Google's Multilingual Neural Machine Translation System*. 2016. Acessado em: 3 abr. 2026. Disponível em: <<https://research.google/blog/zero-shot-translation-with-googles-multilingual-neural-machine-translation-system/>>. Citado na página 14.
- TAN, Z. et al. Neural machine translation: A review of methods, resources, and tools. *AI Open*, Elsevier, v. 1, p. 5–21, 2020. Citado na página 15.
- TEAM, N. et al. *No Language Left Behind: Scaling Human-Centered Machine Translation*. 2022. Disponível em: <<https://arxiv.org/abs/2207.04672>>. Citado na página 26.
- TORREGROSA, D. et al. Leveraging rule-based machine translation knowledge for under-resourced neural machine translation models. In: *Proceedings of machine translation summit XVII: translator, project and user tracks*. [S.l.: s.n.], 2019. p. 125–133. Citado na página 7.
- VASWANI, A. et al. Attention is all you need. *CoRR*, abs/1706.03762, 2017. Disponível em: <<http://arxiv.org/abs/1706.03762>>. Citado nas páginas 14, 15, 20, 22 e 23.
- WANG, H. et al. Progress in machine translation. *Engineering*, Elsevier, v. 18, p. 143–153, 2022. Citado na página 13.
- WANG, X. et al. Neural machine translation advised by statistical machine translation. In: *Proceedings of the AAAI conference on artificial intelligence*. [S.l.: s.n.], 2017. v. 31, n. 1. Citado nas páginas 11, 12 e 14.
- WIKIPEDIA. *Rule-based machine translation* — *Wikipedia, The Free Encyclopedia*. 2026. <<http://en.wikipedia.org/w/index.php?title=Rule-based%20machine%20translation&oldid=1286745997>>. [Online; accessed 02-April-2026]. Citado na página 11.

ZHANG, T. et al. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*, 2019. Citado na página 28.